

面向 AI 原生系统的智能运维

第 17 讲 | 智能体辅助的可观测数据分析与根因定位

从“生成一个答案”到“完成一次可验证的故障调查”

陈鹏飞 | 中山大学
校企联合研究生课程 2026 秋

开场：为什么“把全部日志交给大模型”通常诊断不好？

看起来很强

- ▶ 输入 10 万行日志、几十条指标和完整 Trace；
- ▶ 模型生成流畅的故障故事；
- ▶ 最后给出“缓存、网络或数据库”之一。

实际上不可审计

- ▶ 不知道哪些数据真的被使用；
- ▶ 不知道是否查错时间窗和对象；
- ▶ 没有竞争性假设、反证和停止条件；
- ▶ 症状很容易被改写成根因。

本讲主张 RCA Agent 的核心不是“读更多文本”，而是主动选择高价值观测、维护可证伪假设，并让每一步查询改变后续决策。

课程大纲中的本讲任务

第 17 周要求

讲解智能体如何围绕故障现象主动调用指标、日志、链路、事件和对象档案等工具，逐步收集证据、形成诊断路径、判断根因候选并生成处置建议，理解 Agent Loop 在 RCA 场景中的完整工作过程；课程总结与综合考核。

输入 故障现象、对象图、Episode、观测工具、权限和预算。

过程 假设—查询—观察—更新—反证—停止。

输出 候选、证据、未知、建议、验证条件和调查 Trace。

边界：第 13 讲与第 17 讲不是同一内容

章节	核心问题	主要产物	本讲处理方式
第 13 讲	如何开发一个 RCA Agent 原型？	Task Envelope、状态机、工具 schema、审批/回滚	本讲直接使用，不重复平台搭建
第 15 讲	系统里有哪些对象和关系？	Canonical Object、对象图、来源与版本	作为调查范围和结构先验
第 16 讲	这些对象发生了什么？	Canonical Event、Episode、对象历史记忆	作为时间线和历史先验
第 17 讲	现在应该查什么，怎样证明？	调查策略、竞争假设、主动查询、反证和结论	本讲核心

非目标 不把模型生成文本等同于根因，不把论文准确率直接写成生产 SLA，也不在证据不足时自动执行高风险动作。

学习目标：学生应能独立完成一次可验证调查

1. 把故障现象转化为范围明确、可停止的 Investigation Contract;
2. 为指标、日志、Trace、Profile、事件、对象档案和变更设计查询策略;
3. 使用假设账本、贝叶斯更新、信息增益和查询成本选择下一步;
4. 识别幻觉解释、探索不完整、症状当根因、时间错位和工具噪声;
5. 读懂 RCACopilot、Xpert、OpsLens、HERO、MetaRCA、AIOpsLab、AgentTrace 的机制和结果;
6. 设计涵盖结果、过程、证据、安全和成本的 RCA Agent 评测;
7. 对一个 AI 推理服务故障生成候选、证据、处置建议和终态验证方案。

2 学时路线：沿一条 Investigation Trace 展开



- ▶ 前半段：任务、工具与调查算法，包含公式和数值算例；
- ▶ 中段：多 Agent、Agent 可观测性、失败模式与论文结果；
- ▶ 后半段：LLM 推理服务贯穿案例、实验、综合考核与课程总结。

课堂主线与拓展阅读：先完成一次调查，再扩展论文地图

层次	建议时间	页面范围与教学目标
主讲	70 分钟	1-54 页：调查合同、证据工具、竞争假设、贝叶斯更新、信息增益、停止与升级
选择讲解	20 分钟	55-107 页：多智能体、智能体可观测性、GPU/训练/推理/网络诊断与 RCA 系统；每类只选一个代表方法
贯穿案例	20 分钟	108-123 页：完整走通 TTFT 退化的“假设—查询—反证—处置—验证”链
考核说明	10 分钟	124-141 页：结果、轨迹、证据、安全、成本与课程实验；142-145 页作为参考文献和结束页

时间控制 训练诊断、在线推理、网络 and RCA Agent 各选择一条最相关证据链，其余作为论文阅读材料。

一、调查合同与世界状态

先定义要证明什么，再让 Agent 查询系统

RCA 的产物不是一个标签，而是一条可审计论证链



- ▶ 根因候选必须说明：故障对象、机制、开始时间、传播路径和影响；
- ▶ 每个字段都要能回到观测证据或明确标记为推断；
- ▶ 最终结论要包含仍未解释的现象，以及什么新证据会推翻它。

Investigation Contract: 一次调查的最小 Schema

字段	示例	作用
incident / symptom	TTFT P95 从 1.2s 升至 4.8s	定义要解释的结果变量
scope	endpoint/model-v3/region-a	限制对象和租户边界
time window	09:00-09:30, 含 30min baseline	保证变更与观测可对齐
available tools	metrics/logs/traces/events/profile	决定可获取证据
constraints	read-only, 12 次调用, 10 分钟	控制风险、成本和停止
done when	候选 + 支持 + 反证 + 验证方案	防止“有一个像答案的句子”就结束

工程原则 Contract 在运行开始时冻结；若范围改变，应产生新版本并记录是谁、依据什么改变。

完成条件：什么叫“诊断结束”？

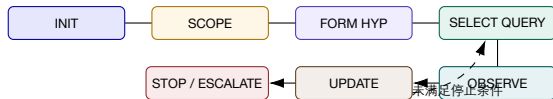
最小结论 至少一个候选，绑定对象、机制、时间和证据。

可信结论 存在独立模态交叉验证，关键反例已检查。

可处置结论 有最小动作、风险、回滚和终态 Oracle。

- ▶ “查询次数用完”是预算终止，不是诊断成功；
- ▶ “没有发现异常”只说明当前工具/窗口未观察到，不等于没有故障；
- ▶ 允许输出 UNKNOWN、CONTESTED 或 NEEDS_HUMAN；
- ▶ 严重事故宁可保留两个候选，也不要低置信推断包装成唯一根因。

Agent Loop 的状态不是一段隐藏的“思考文本”



每个状态都保存输入版本、输出、耗时、失败、预算变化和 causation ID；模型文本只是其中一项，不是状态本身。

World State: Agent 每一轮共享同一个“当前世界”

状态域	内容	更新规则
Scope	对象、租户、区域、版本、时间窗	只能由显式证据或人工扩大
Object Graph	调用、部署、资源、模型、数据关系	带 valid time 与 provenance
Episode Timeline	事件、变更、影响、已执行动作	append-only, 允许 supersedes
Hypothesis Store	候选、先验、支持、反证、未知	每次观察后版本化更新
Evidence Ledger	查询、原始结果、摘要、覆盖和质量	原始证据不可被摘要覆盖
Budget/Policy	Token、时间、调用数、权限、风险	每个动作前重新检查

常见错误 并行 Agent 各自维护一份无版本世界状态，会产生“对象不同、时间窗不同、证据相互覆盖”的伪协作。

证据、推断和动作：三层不能混写

Evidence

- ▶ 09:06 TTFT P95=4.8s;
- ▶ trace-91 的
prefill=3.1s;
- ▶ model-v3 于 08:58 发布。

Inference

- ▶ 发布与退化时间相符;
- ▶ prefill 是主要延迟贡献;
- ▶ 新模型可能改变显存压力。

Decision

- ▶ 查询 batch/KV/profile;
- ▶ 影子回滚 model-v2;
- ▶ 暂不重启整个服务。

写作规则 事实句引用 evidence ID; 推断句列出假设和置信; 动作句绑定 policy、owner、rollback 与 oracle。

Hypothesis Record: 候选本身也有 Schema

字段	示例
hypothesis	H1: model-v3 导致 KV cache 压力, 进而抬高 TTFT
mechanism	context length↑ → KV block 占用↑ → waiting↑ → prefill 排队↑
scope/time	endpoint-a, model-v3, 08:58 之后
prior / posterior	0.30 / 0.62 (当前证据后)
supports	e17 发布时间、e22 KV 93%、e25 waiting 12
contradicts	e31 model-v2 也出现过一次同类峰值
unknowns	batch policy、请求长度分布、GPU-3 profile
next best test	比较同负载 model-v2/v3 的 KV 和 prefill span
status	active / weakened / disproved / verified / contested

Claim-Evidence Graph: 为什么比长文本更适合 RCA?



- ▶ 同一证据可以支持一个候选、反驳另一个候选；
- ▶ 图结构允许查询“哪个结论只有单一来源”“哪个反证未处理”；
- ▶ 摘要由图生成，但图中仍保留原始 evidence reference。

不确定性要分开表达：概率与证据质量不是一回事

Likelihood

- ▶ 候选相对可能性；
- ▶ 可用 posterior 或低/中/高区间；
- ▶ 随新证据变化。

Confidence

- ▶ 证据来源、覆盖和一致性；
- ▶ 多源独立验证通常提高 confidence；
- ▶ 高概率也可能建立在低质量数据上。

规范表达

“H1 当前较可能 (posterior 0.62)，但证据置信度中等：KV 与 waiting 同步上升，尚缺同负载版本对照；若 model-v2 对照同样退化，H1 将显著减弱。”

RCA 输出合同：结论必须能被人 and 系统继续消费

输出块	必需内容
Observed impact	用户/租户/区域、SLO、开始时间、证据覆盖
Ranked candidates	对象 + 机制 + 范围 + 概率/等级，至少保留第二候选
Evidence matrix	supports、contradicts、unknowns、来源与时间
Investigation trace	每次 query 的目标、参数、结果、状态变化和成本
Recommendation	最小动作、风险、审批者、回滚、终态 Oracle
Residual uncertainty	仍解释不了什么、哪些数据缺失、何时需人工
Memory update	可沉淀事实、模式候选、被推翻假设和适用条件

一次调查可能怎样失败？先把失败写进设计

数据失败 没有事件、采样偏差、时间错位、对象身份错误、观测被裁剪。

推理失败 症状当根因、确认偏误、单一假设、虚构解释、忽略反例。

系统失败 工具超时、权限拒绝、并发冲突、上下文耗尽、执行环境 OOM。

- ▶ 每种失败都必须映射到可观测状态，而不是统一返回空字符串；
- ▶ 结果错误与过程错误分别评测：答错、查错、没查、查了但误解；
- ▶ 修复 Agent 失败时，先判断该改 Prompt、工具、协议还是环境。

贯穿案例初始化：推理服务 TTFT 退化

已知事实

- ▶ 09:03 起 TTFT P95 1.2s→4.8s;
- ▶ TPOT 基本稳定, 5xx 未显著上升;
- ▶ 08:58 发布 model-v3;
- ▶ 影响长上下文请求更明显。

初始候选

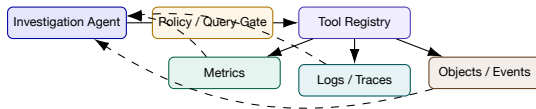
- ▶ H1: KV cache / batch 排队;
- ▶ H2: 模型权重或 kernel 退化;
- ▶ H3: 跨机网络/collective 退化;
- ▶ H4: 上游 RAG 工具延迟。

此刻不能做的事 仅凭“发布后变慢”就宣布 model-v3 是根因；部署时间只是高价值先验，不是因果证明。

二、可观测工具与证据数据 面

工具返回的不是字符串，而是带范围、质量和失败语义的证据

工具平面：从“查数据”到“回答一个诊断问题”



Gate 负责范围、权限、成本、速率、参数和脱敏；Registry 负责版本、schema、SLO 与失败语义；Agent 只在合同允许的空间里选工具。

Tool Contract: 六项缺一不可

合同项	示例	为什么重要
diagnostic intent	比较 v2/v3 的 prefill latency	描述要区分哪个假设
input schema	object, window, baseline, group-by	防止对象和时间漂移
output schema	values, coverage, evidence IDs	便于验证和组合
failure semantics	empty/denied/timeout/partial/error	避免把“没拿到”当“没有异常”
cost/risk	latency, rows, token, permission	进入查询价值计算
freshness/version	watermark, query/tool version	支持重放与数据新鲜度判断

指标工具：一个点值通常不足以支持 RCA

- ▶ 请求必须包含对象、标签过滤、事件窗、基线窗、聚合和分位数；
- ▶ 返回值必须说明采样间隔、缺失率、基数裁剪和时钟水位；
- ▶ 同时报 change point、effect size 与基线差，不只报“超过阈值”；
- ▶ 对 AI 推理至少分解 queue、prefill、decode、KV、batch、GPU kernel；
- ▶ 对训练至少分解 job/step/rank/collective、checkpoint 和 straggler。

示例返回

query: ttft_p95 by model_version window: 09:00–09:15 baseline: 08:00–08:30
result: v3 +286%, v2 +8% coverage: 98.7% evidence: metric-44

日志工具：返回“模式差异”，而不是倾倒原文

1. 先限定对象、版本和时间窗，解析模板/事件类型；
2. 比较故障窗与基线窗的新增、消失和频率突变模板；
3. 返回代表样本、计数、首次/末次出现、来源和上下文；
4. 高严重度原文可以附带，但不能替代结构化差异；
5. PII、密钥、Prompt 和客户内容必须脱敏并记录 redaction。

例子 “出现 3,421 行 warning” 信息量很低；“model-v3 上新增 kv allocation retry, 09:02 首次出现，与 waiting 上升同步” 才可支持假设。

Trace 工具：比较路径、阶段和等待，而不是只找红色 Span

请求切片

- ▶ 正常/异常、v2/v3;
- ▶ 短/长上下文;
- ▶ region、tenant、request class;
- ▶ 同一上游和相近负载。

返回差异

- ▶ 路径新增/缺失;
- ▶ span duration 与 self time;
- ▶ queue/wait/retry;
- ▶ error boundary 与采样覆盖。

反例 异常 Trace 的最后一个慢 Span 可能只是下游等待的表现；必须沿依赖和时间反向查到最早偏离。

Profile 工具：回答“时间花在哪里”，但不能自动回答“为什么”

- ▶ CPU profile: on-CPU/off-CPU、锁、调度、系统调用；
- ▶ GPU profile: kernel、memory、stall、launch gap、通信 overlap；
- ▶ continuous profile 要绑定 service/pod/model version/request class；
- ▶ 比较 profile 时必须控制负载、batch、输入长度和硬件；
- ▶ profile 热点是机制证据；根因还需时间、变更、对象和反事实支持。

查询设计

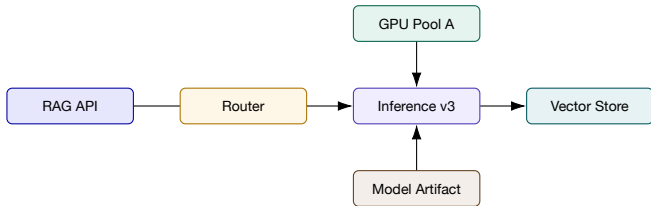
若 H2 预测 model-v3 的 attention kernel 变慢，则比较同 GPU、同长度、同 batch 的 v2/v3 kernel duration；若两者无差异，H2 应明显下降。

事件与记忆工具：历史相似不是复制答案

- ▶ Episode 检索要按对象版本、症状、事件模式、负载条件和拓扑 motif;
- ▶ 返回相似度之外，还要返回差异字段和旧结论的验证状态;
- ▶ 历史处置仅作为 procedural prior，必须检查 TTL、适用版本和副作用;
- ▶ 被推翻的历史假设也应检索出来，帮助 Agent 避免重复弯路;
- ▶ 新事件没有历史匹配时，应提高 unknown，而不是强行选择最近邻。

承接第 16 讲 对象记忆提供“过去在什么条件下发生过什么”；本讲 Agent 负责判断当前条件是否真的相同。

对象图与拓扑工具：提供结构先验，也要带时态



- ▶ 询问 09:03 时刻的关系，不应读取 10:00 已更新的当前拓扑；
- ▶ ownerReference、Service selector、Trace edge 和部署关系证据等级不同；
- ▶ 拓扑用于限制候选与传播路径，不能单独证明因果。

变更工具：时间相近是先验，不是判决

变更类型	应返回的上下文	可区分的假设
deploy/model	artifact、config、rollout、owner、diff	代码/模型/参数退化
feature flag	scope、percentage、tenant、effective time	局部影响与灰度差异
autoscaling	desired/actual、admission、placement	资源不足或调度变化
network/policy	route、ACL、ToR/plane、endpoint	路径故障与连接失败
data/prompt	index、corpus、template、schema version	RAG/质量故障

反证 如果未变更实例也同样退化，或变更发生在症状之后，则“最近发布”候选必须降权。

知识/RAG 工具：返回可引用材料，而不是“最佳实践答案”

- ▶ 查询对象、症状、机制和当前版本，返回 runbook、设计文档、postmortem；
- ▶ 每段材料携带文档 ID、版本、更新时间、owner 和适用对象；
- ▶ 检索结果中的命令、SQL 或配置不得直接执行；
- ▶ 旧文档与当前对象冲突时，以运行时证据为准并标记 stale；
- ▶ 检索不到材料只说明知识覆盖不足，不能削弱一个由运行时证据支持的候选。

分工 知识提供机制和查询提示；观测提供当前事实；Verifier 检查二者是否在同一对象、版本和时间上成立。

生成查询必须通过静态和动态双重验证

执行前

- ▶ schema/table/label 是否存在；
- ▶ 类型、时间单位和操作符是否正确；
- ▶ scope、limit、timeout 是否满足策略；
- ▶ 是否读取敏感字段或跨租户。

执行后

- ▶ 覆盖率、行数和 watermark；
- ▶ 返回是否回答了原诊断问题；
- ▶ error/empty/partial 的真实含义；
- ▶ 是否需要重试、缩小或人工确认。

Xpert 的启示是：语法可执行仍不等于语义正确；表、过滤条件、聚合和输出列都要进入验证。

工具失败语义：空结果至少有五种不同解释

状态	含义	Agent 应如何更新
EMPTY	查询成功，范围内确实无记录	可作为有限反证，但检查覆盖
PARTIAL	部分 shard/对象缺失	不进行强结论，补查缺口
DENIED	权限或策略阻止	证据 unknown，申请权限/人工
TIMEOUT	计算或后端超时	缩小范围、换摘要接口，不当作 empty
SCHEMA_ERROR	字段/版本不匹配	修正查询，记录工具/Agent 缺陷
STALE	数据水位落后事件窗	等待水位或换数据源

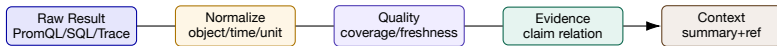
时间对齐：Agent 最容易犯、也最便宜修的错误

- ▶ 明确 event time、ingest time、query time 和 clock offset；
- ▶ 每次查询同时保存绝对时间与相对 incident time；
- ▶ 变更、指标、日志和 Trace 使用同一时区/单调序列；
- ▶ 对延迟到达数据设置 watermark，更新后允许重开假设；
- ▶ 比较基线时避免把日周期、工作日/周末和负载变化混入。

典型陷阱

日志存 UTC、告警 UI 显示 CST、变更单用本地时间；Agent 把相差 8 小时的发布当成故障前一分钟发生。

证据规范化：异构模态先对齐，再进入上下文



- ▶ 单位统一：ms/s、bytes/GiB、rate/count；
- ▶ 身份统一：pod UID、service、model version、rank；
- ▶ 时间统一：window、baseline、offset、watermark；
- ▶ 关系统一：supports/contradicts/context/unknown。

Context Budget: 保留能改变决策的内容

- ▶ Tier 0: 任务、范围、当前候选、关键证据和安全约束；
- ▶ Tier 1: 最新观测摘要、反证、未解决问题和下一步候选；
- ▶ Tier 2: 可按 evidence ID 检索的原始数据和长文档；
- ▶ 已被证明重复或无关的数据移出 **working context**，但不删除 **ledger**；
- ▶ 压缩时保留数值、时间、对象、方向、异常/基线和引用。

判断标准 若删除某条内容不会改变候选排序、下一查询或停止判断，它通常不应常驻模型上下文。

Evidence Bundle：一次工具结果怎样进入 Agent?

字段	示例
intent	区分 H1 KV 排队与 H2 kernel 退化
query	compare kv_usage, waiting, prefill by version
scope/time	endpoint-a; 08:50-09:20; baseline 08:00-08:30
observation	v3 KV +41%, waiting 0→12; kernel duration +3%
quality	coverage 98.7%; watermark 09:19:50; 同 GPU 对照
supports/contradicts	supports H1 0.8; contradicts H2 0.6
raw refs	metric-44, trace-91, profile-18
limitations	长上下文样本量 47; v2 负载略低 8%

Coverage Matrix: 查了很多，不代表覆盖了关键空间

候选	Metrics	Logs	Trace	Change	未覆盖
H1 KV/queue	yes	yes	yes	yes	同负载 v2/v3 对照
H2 kernel	partial	-	yes	yes	GPU profile/kernel
H3 network	yes	-	yes	-	RNIC/path/plane
H4 RAG tool	yes	yes	yes	-	retrieval quality/timeout

- ▶ 覆盖按“候选 × 证据模态 × 对象范围”计算；
- ▶ 同一日志工具调用 10 次不能替代未查询的 Trace/Profile；
- ▶ Stop 前要检查最高候选的关键预测是否至少被一个独立模态验证。

Agent-Cloud Interface: 接口设计决定调查上限

面向人的接口

- ▶ Dashboard、自由查询、上下文靠经验；
- ▶ 允许视觉跳转和隐含状态；
- ▶ 错误提示常为自然语言。

面向 Agent 的接口

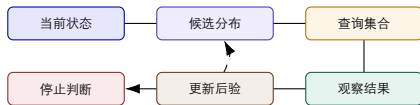
- ▶ 明确 schema、范围、状态和 cost；
- ▶ 可发现、可组合、可验证、可重放；
- ▶ 失败类型、coverage 与 provenance 结构化。

工程结论 同一个模型，使用“返回十万行文本”的工具和“返回可比较 Evidence Bundle”的工具，诊断能力会完全不同。

三、竞争性假设与主动查询

下一步不是“再查一点”，而是选择最能区分候选的证据

证据驱动的 Agent Loop



每一轮三个问题

当前哪些候选仍能解释所有关键现象？哪一条查询最可能区分它们？查询结果如何改变候选、未知和下一步？

生成假设：从故障机制出发，不从组件列表抽签

1. 以结果变量为起点：TTFT 上升但 TPOT 稳定；
2. 沿请求阶段分解：admission → queue → prefill → decode → tool；
3. 对每阶段列机制：容量、配置、资源、软件、网络、数据；
4. 用对象图限定实际存在的组件和依赖；
5. 用 Episode/变更设置先验，但保留 novel/unknown；
6. 为每个候选写出“若为真，应观察到什么；若为假，什么会出现”。

反模式 候选只有“GPU、网络、数据库”三个名词，没有机制和可验证预测，Agent 只能收集相关异常，无法证伪。

问题分解：按阶段、对象、时间和影响切片

阶段 请求在哪个阶段首次变慢？

对象 哪些版本、Pod、GPU、租户不同？

时间 变化先于还是晚于症状？

影响 所有请求还是某类输入？

- ▶ 阶段切片缩小机制空间；对象切片构造自然对照组；
- ▶ 时间切片排除症状之后的伴随变化；
- ▶ 影响切片揭示 **feature flag**、租户、模型版本和请求长度条件；
- ▶ 四种切片组合后，查询才从“看 dashboard”变成检验假设。

搜索空间：先剪掉不可能，再排序可能

- ▶ Scope pruning: 不在影响对象和有效拓扑内的候选先排除；
- ▶ Temporal pruning: 变化发生在症状之后，不能作为初始原因；
- ▶ Mechanism pruning: 无法产生该症状组合的机制降权；
- ▶ Coverage pruning: 没有对应观测时标为 unknown，不误判为 false；
- ▶ Risk pruning: 需要高风险动作才能验证的候选，先寻找只读替代测试；
- ▶ Diversity: 保留至少一个不同机制的候选，防止所有候选共享同一错误前提。

例子 TPOT 稳定而 TTFT 上升，使 decode kernel 退化的先验下降，但不能完全排除共享 GPU 干扰；需 Profile/queue 对照进一步区分。

先验从哪里来？四类来源需要分级

来源	先验作用	风险
对象/拓扑 Episode 历史 近期变更 领域知识/LLM	限制实际依赖和故障域 相似条件下的复发模式 提高时间相近候选优先级 生成机制和可检验预测	图过期、声明关系 $\neq$ 运行关系 版本/负载不同、幸存者偏差 变更偏见、并发变化 过度泛化、幻觉因果

原则 先验决定“先查谁”，不能决定“谁就是根因”；任何高先验候选仍需当前事故证据。

贝叶斯更新：证据的作用是改变候选相对概率

后验赔率

$$\frac{P(H_i | E)}{P(H_j | E)} = \frac{P(H_i)}{P(H_j)} \times \frac{P(E | H_i)}{P(E | H_j)}$$

- ▶ 似然比表达 “这条证据在不同候选世界里有多常见”；
- ▶ 证据不是独立时不能重复相乘，例如 **alert** 和同指标日志可能同源；
- ▶ 工程上可使用离散权重/打分，但要保留方向和依赖关系；
- ▶ 后验用于排序调查，不应伪装成经过校准的生产概率。

数值算例：一条版本对照怎样改变 H1 与 H2?

初始先验

$P(H_1=KV) = 0.40$, $P(H_2=kernel) = 0.35$, 其他候选 0.25。

观察 E

同负载下 v3 的 KV/waiting 明显上升，而 kernel duration 仅 +3%。估计 $P(E | H_1) = 0.80$, $P(E | H_2) = 0.15$ 。

$$\text{Odds}'_{1:2} = \frac{0.40}{0.35} \times \frac{0.80}{0.15} = 6.10$$

解释 H1 相对 H2 的赔率从 1.14 上升到 6.10；这足以优先验证 H1，但仍未排除“流量构成变化”这一替代解释。

矛盾证据如何处理？不要用平均分把冲突抹平

- ▶ 先核对对象、时间、单位、采样和数据来源是否一致；
- ▶ 区分“同一事实冲突”和“不同条件下结果不同”；
- ▶ 给出条件化候选，例如 H1 只在长上下文 +batch>8 时成立；
- ▶ 如果两条高质量独立证据仍冲突，状态设为 CONTESTED；
- ▶ 选择能区分条件的下一查询，而不是让 LLM 写一个折中故事。

例子

GPU-3 KV 93%，GPU-7 仅 62%。先检查请求路由、模型副本、context length 与卡型；平均成 77.5% 会丢掉真正的故障域。

信息增益：下一条查询应最大幅度减少不确定性

期望信息增益

$$IG(q) = H(\mathbf{H}) - \mathbb{E}_{o \sim q}[H(\mathbf{H} \mid o)], \quad H(\mathbf{H}) = - \sum_i p_i \log_2 p_i$$

- ▶ 查询结果若在所有候选下都相同，信息增益接近 0；
- ▶ 能把候选分成不同预测的对照实验通常 IG 高；
- ▶ 实际调度还要减去延迟、Token、计算和风险成本；
- ▶ 没有精确概率时，可用“预期区分候选数 × 证据质量”近似。

信息增益小算例：先查 **Profile**，还是先查更多日志？

当前分布

$P(H_1) = 0.50$, $P(H_2) = 0.35$, $P(H_3) = 0.15$, 熵约 1.44 bits。

- ▶ q_1 更多同类日志：预计结果在 H1/H2 都会出现，后验熵约 1.30, $IG \approx 0.14$;
- ▶ q_2 v2/v3 GPU Profile：能直接区分 queue 与 kernel，后验熵期望约 0.72, $IG \approx 0.72$;
- ▶ 若 q_2 需 40 秒、 q_1 需 3 秒，可比较 $IG/time$ ，但严重事故也要考虑结论价值。

选择 已有日志已覆盖 H1 机制，重复查询边际价值低；Profile 虽更贵，却更可能改变排序，应优先。

查询效用：信息、成本、风险和等待共同决定顺序

教学评分函数

$$U(q) = \alpha I G(q) + \beta D(q) + \gamma C_v(q) - \lambda L(q) - \mu Cost(q) - \nu Risk(q)$$

- ▶ D ：对当前最高候选的区分力； C_v ：交叉验证价值；
- ▶ L ：延迟； $Cost$ ：Token/扫描量/计算； $Risk$ ：权限与副作用；
- ▶ 只读、缓存、低成本、高区分力查询优先；
- ▶ 高风险干预只有在只读证据不足且收益足够时才进入审批。

系数不是通用常数，应按严重度、SLO 和组织风险偏好配置，并通过回放校准。

串行调查：当前一步会改变下一步时，先观察再决定

1. 查分阶段 Trace，确定问题集中在 prefill；
2. 因此不再查 decode 大量日志，转向 queue/KV；
3. 查版本对照，发现 v3 独有；
4. 再查 Profile 区分 kernel 与等待；
5. 只有 H1 足够强时，才提出影子回滚验证。

优点 每一步减少后续查询空间，避免把所有 dashboard 一次性扫描；适合依赖强、预算紧、风险高的 RCA。

并行调查：独立、只读且时间敏感的查询可以同时执行

适合并行

- ▶ 指标基线、Trace 对照、变更查询；
- ▶ 不共享可变状态；
- ▶ 结果能按 evidence ID 合并；
- ▶ 后端有速率和成本余量。

不适合并行

- ▶ 后一步参数依赖前一步结果；
- ▶ 多次扫描同一昂贵数据；
- ▶ 会产生相互干扰的写操作；
- ▶ 汇总者无法处理冲突和版本。

协调要求 并行返回必须带同一 investigation version；晚到结果不能静默覆盖已做出的停止/动作决策。

停止条件：不是“模型感觉已经够了”

- ▶ 证据停止：最高候选满足最小支持与独立交叉验证；
- ▶ 区分停止：剩余查询的期望效用低于阈值；
- ▶ 风险停止：验证需要超出权限或不可逆动作；
- ▶ 预算停止：时间/Token/调用上限到达，输出 `incomplete`；
- ▶ 冲突停止：关键证据相互矛盾，需要人工或新数据；
- ▶ 安全停止：工具返回异常、`Prompt injection` 或范围漂移。

成功条件 “停止” 必须同时输出原因、当前最佳候选、缺失证据和恢复调查的触发条件。

Unknown 与 Escalation: 可信 Agent 必须会承认不知道

UNKNOWN

- ▶ 观测覆盖不足;
- ▶ 候选均不能解释关键现象;
- ▶ 新故障模式, 无历史/知识支持;
- ▶ 数据质量不足以排序。

NEEDS HUMAN

- ▶ 需要跨权限域查询;
- ▶ 严重度高且候选冲突;
- ▶ 验证需要业务取舍;
- ▶ 工具/环境持续失败。

升级包应包含已查证据、未查空间、失败工具、候选变化和建议人工问题, 不能只写“请联系专家”。

Verifier: 不仅检查 JSON, 还检查论证结构

1. Schema: 字段、类型、对象、时间和 evidence ID 合法;
2. Entailment: 每个事实句能从引用证据推出;
3. Temporal: 原因候选变化不晚于症状;
4. Coverage: 最高候选的关键预测已检查;
5. Counterevidence: 重要反证没有被摘要隐藏;
6. Policy: 建议动作、权限、回滚和 oracle 匹配;
7. Calibration: 语言强度与证据质量一致。

边界 Verifier 也可能错; 关键断言应使用确定性规则、查询重放或人工复核, 而不是再让一个模型“觉得合理”。

“事后故事” 陷阱：解释连贯不等于因果正确

错误写法

“model-v3 发布后显存升高，因此新模型导致 TTFT 退化，回滚即可解决。”

证据写法

“v3 发布先于 TTFT 退化 5 分钟；同负载 v3 的 KV/waiting 显著高于 v2，kernel 基本一致。该证据支持 H1；尚缺影子回滚终态验证。”

- ▶ 前者隐藏了对照、条件、反例和未验证动作；
- ▶ 后者允许别人定位原始证据并提出反驳；
- ▶ 高质量 RCA 是可被推翻、可被更新的论证，不是文学作品。

示例 Investigation Trace： 每一步都改变了什么？

时间	查询意图	观察	状态变化
09:05	分解 TTFT 阶段	prefill wait +2.7s, decode 稳定	H1/H2↑, H4↓
09:07	版本/长度对照	v3+ 长上下文最明显	H1↑, 限定条件
09:09	GPU profile	kernel +3%, launch gap 稳定	H2↓
09:11	KV/waiting	KV 93%, waiting 12; v2 低	H1↑ 至主候选
09:13	RNIC/path	丢包/RTT/重传正常	H3↓
09:15	反例搜索	v2 一次峰值由流量突增解释	H1 保留, 补负载条件

下一步 提出只读/影子对照或受控回滚；在终态验证前，状态仍是 SUPPORTED 而非 VERIFIED。

四、多智能体调查与 Agent 可观测性

让专业 Agent 协作，但让证据和责任仍然可追踪

为什么要多 Agent：并行的是证据空间，不是答案数量

单 Agent 的瓶颈

- ▶ 指标、日志、Trace、GPU、网络的查询语言不同；
- ▶ 长上下文使早期证据被压缩或遗忘；
- ▶ 同一模型提出并验证假设，确认偏误更强；
- ▶ 工具失败容易被误解为“没有异常”。

多 Agent 的真正收益

- ▶ 专业化查询与并行收集；
- ▶ 独立 Verifier 挑战主候选；
- ▶ Coordinator 统一范围、预算和停止条件；
- ▶ 结构化交接降低上下文搬运成本。

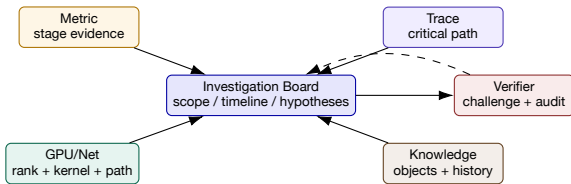
误区 五个 Agent 投票同一个未经核验的故事，不会产生五倍可信度；独立证据源和可证伪预测才增加可信度。

角色设计：按认知职责分工，而不是按模型名称分工

角色	核心输入	主要输出	不允许做什么
Coordinator	Contract、候选、预算	下一步查询、停止/升级	私自扩大范围
Metric Agent	SLI、资源、基线	阶段分解、异常区间	仅凭相关性定根因
Log/Trace Agent	模板、span、causation ID	错误序列、关键路径	复制海量原文
GPU/Network Agent	kernel、rank、collective、path	慢点、故障域、对照	将等待时间当执行时间
Knowledge Agent	对象、变更、历史 Episode	先验、相似与差异	把 runbook 当当前证据
Verifier	Claim、evidence bundle	entailment、反证、边界	只检查格式

分工原则 每个专业 Agent 返回相同 Evidence Bundle；Coordinator 不读工具私有格式，只消费契约化证据。

共享调查板：协作的中心是状态，不是群聊



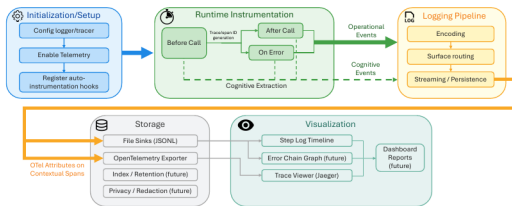
- ▶ Board 只保存版本化 Claim、Evidence、Question、Decision;
- ▶ Agent 通过 optimistic concurrency 或单写者更新候选分数;
- ▶ 冲突不覆盖：保留 supports/contradicts/supersedes 边。

Agent 间消息也必须是可执行契约

字段	示例	检查
question	H3: RNIC/path 是否导致 collective tail?	指向一个可区分假设
scope	ranks 32-63, 09:02-09:08	不越租户/对象/时间
requested evidence	per-hop RTT, retransmit, ECN, QP error	不是“查一下网络”
result	p99 RTT stable; rank 47 QP retry spike	事实与摘要分开
evidence refs	net-21, rdma-08	可重放、带查询参数
confidence/limits	medium; ToR counter missing 90s	表达覆盖与缺口
next discriminator	compare rank47 peer traffic	给 Coordinator 决策输入

自然语言适合解释；对象、时间、证据、状态和权限必须结构化。

AgentTrace: 要观测“认知—执行—上下文”三个表面



- ▶ **Cognitive:** 目标、计划、决策与假设变化；
- ▶ **Operational:** 工具调用、延迟、重试、错误与成本；
- ▶ **Contextual:** 输入来源、检索、记忆、模型与配置版本；
- ▶ **JSONL/OTel** 只是载体，关键是稳定 schema 与关联 ID。

来源: AgentTrace: A Structured Logging Framework for Agent System Observability, arXiv:2602.10133, Fig. 1.

Agent Trace Schema：一次“为什么查错了”需要哪些字段？

层面	必备字段	可回答的问题
Decision	goal, hypothesis IDs, selected action, alternatives	为什么查这个，而不是另一个？
Tool	name/version, args hash, result status, latency	是系统无异常，还是查询失败？
Evidence	source, object, event/valid time, quality, refs	结论能否回到原始数据？
Model	provider/model/prompt version, tokens, temperature	模型变化是否改变诊断路径？
Context	retrieved docs, memory IDs, truncation	是否被旧知识或截断上下文误导？
Policy	authorization, approval, redaction, action risk	是否越权或泄露敏感数据？
Outcome	stop reason, result state, oracle	是成功、未知还是预算终止？

隐私边界 不默认记录完整 chain-of-thought、Secrets 或原始用户数据；记录可审计的决策摘要和证据引用即可。

AgentTrace-RCA: 从结构化事件反向追踪失败传播

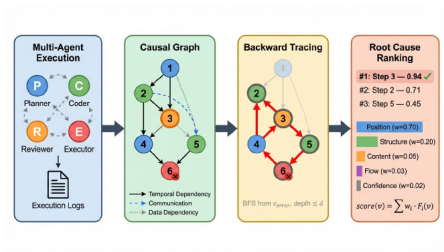


Figure 1: Overview of the AGENTTRACE framework. (a) Execution logs from a multi-agent work-flow are collected. (b) A causal graph is constructed with sequential, communication, and data dependencies. (c) Backward tracing identifies the root cause event. (d) Root cause ranking identifies the most likely root cause event.

1. 把 Agent 事件构造成因果/依赖图；
2. 从失败 Outcome 沿依赖边反向传播；
3. 汇聚对终态最有解释力的上游事件；
4. 输出 ranked root-cause event，而非自由文本故事。

来源：Wang, *AgentTrace: Causal Graph Tracing for Root Cause Analysis in Deployed Multi-Agent Systems*, ICLR 2026 Workshop on Agents in the Wild, Fig. 1.

AgentTrace-RCA 结果：高 Hit@1，但问题被严格限定

94.9%

Hit@1, 95% CI 92.9–96.7

98.4%

Hit@3

0.12 s

平均分析时间；LLM baseline 8.3

s

- ▶ 550 个合成场景、10 个领域、每条 8–15 个动作；
- ▶ 每个场景只注入一个根因，结构化日志准确可用；
- ▶ 28 个失败样本主要来自多个合理根因难以区分；
- ▶ 结果证明结构化追踪在该设定有效，不证明生产多根因事故可达 94.9%。

来源：Wang, *AgentTrace: Causal Graph Tracing for Root Cause Analysis in Deployed Multi-Agent Systems*, ICLR 2026 Workshop on Agents in the Wild, evaluation and error analysis.

OpsLens: 把模态专长与工具质量控制组合起来

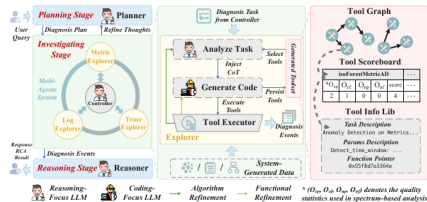


Fig. 3. The overall architecture of OpsLens.

- ▶ 模态专门 Agent 处理不同证据；
- ▶ 工具质量控制过滤无效或低质量返回；
- ▶ Coordinator 汇总为可追踪诊断；
- ▶ runbook-free 不等于 knowledge-free: 对象、工具语义与验证仍必需。

来源: OpsLens: Industrial Runbook-Free Root Cause Analysis with Modality-Specialized Agents and Quality-Controlled Tools, ASE 2026 Industry Track (to appear). 此处不引用未公开实验数字。

共识失败案例：三个 Agent 为什么一起选错？

Agent	看到的证据	结论	隐藏依赖
Metric	TTFT 与 network wait 同时升高	网络故障	network wait 来自同一错误 span
Trace	gateway-engine span 变长	网络故障	span 未分离排队与传输
Log	“transport timeout” 增多	网络故障	timeout 是下游 KV 饱和的结果

- ▶ 三个模态其实共享同一个观测建模错误，证据不独立；
- ▶ Verifier 应提出 RNIC/path counter 与同机版本对照；
- ▶ 如果网络计数正常而 waiting 与 KV 强相关，假设应转向调度/缓存。

调查 Agent 自己也会故障：形成“双层 RCA”



- ▶ 目标系统 RCA：哪个对象/机制造成服务失败？
- ▶ Agent RCA：哪个决策、工具、上下文或交接造成误诊？
- ▶ 二者用不同 causation ID，通过 incident ID 和 evidence refs 关联。

何时不该使用多 Agent?

单 Agent/确定性流程更好

- ▶ 任务只有一个稳定 API 和固定决策树；
- ▶ 故障模式少、查询代价高；
- ▶ 数据高度敏感，跨 Agent 复制风险大；
- ▶ 证据强耦合，无法有效并行。

多 Agent 值得

- ▶ 多模态、多权限域、可独立查询；
- ▶ 候选多且需要并行反证；
- ▶ 专业工具语义差异显著；
- ▶ 可通过统一 schema 审计协作。

工程结论 先证明单 Agent 的瓶颈来自证据空间，再引入多 Agent；否则只是把成本、延迟与故障面成倍放大。

小结：多 Agent 的四个不变量

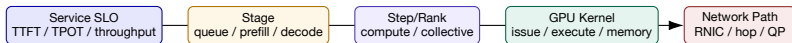
1. 同一个 Investigation Contract：范围和完成条件不可各自解释；
2. 同一个 Evidence Schema：工具结果可比较、可回放、可引用；
3. 同一个 Hypothesis Store：支持、反证和未知不会散落在对话里；
4. 同一个 Safety/Stop Policy：任何 Agent 都不能绕过权限和终态验证。

过渡 接下来不按论文顺序讲，而按调查所需的“证据分辨率阶梯”分析近期系统：服务阶段 → 训练步骤 → rank/collective → GPU kernel → 网络路径。

五、AI 系统诊断的证据分辨率阶梯

论文不是名单，而是回答不同调查问题的工具

先选分辨率：不要一上来就采集所有 GPU kernel



- ▶ 每次下钻都要由上一层证据触发，并产生更可区分的预测；
- ▶ 分辨率越细，采集成本、数据量和侵入风险通常越高；
- ▶ 如果阶段级对照已排除 GPU，就不应继续 kernel profiling；
- ▶ 如果 collective tail 只发生在一组 rank，再查询相关 path，而非扫描全网。

训练慢在哪里？ OSDI'25 用 What-if 构造反事实基线

观察世界

混合 DP/PP/TP/CP 的训练步骤，各 worker 的操作时间不同；同步将局部慢放大为全局慢。

反事实世界

用可比较 worker/操作的非慢值替换 straggler，模拟“若这些慢点不存在”的 JCT：

$$S = \frac{T_{actual}}{T_{ideal}}, \quad W = 1 - \frac{T_{ideal}}{T_{actual}}$$

- ▶ 分别“修复”某类操作、stage 或 worker，估计其可恢复性能；
- ▶ 因而能区分：计算、PP 通信、DP 通信、划分不均，还是少数机器。

来源：Understanding Stragglers in Large Model Training Using What-if Analysis, OSDI 2025, §3–§5.

What-if 研究结果：慢不是偶发尾点，也不总是坏机器

42.5%

训练作业至少慢 10%

10.4%

总体分配 GPU-hours 因
straggler 浪费

1.7%

仅此比例作业的多数 slowdown
由少数慢 worker 解释

- ▶ 超过 10% 的作业至少浪费 21.3% 的 GPU-hours；约 1% 达 45%；
- ▶ 大多数 step 的 slowdown 接近作业总体 slowdown，说明常是持续因素；
- ▶ 计算 slowdown 比通信更常主导；39.3% 的 PP 作业中，最后 stage 解释多数 slowdown；
- ▶ 数据来自五个月、3,079 个作业；网络已充分供给的集群条件限制外推。

来源：Understanding Stragglers in Large Model Training Using What-if Analysis, OSDI 2025, Fig. 3–7.

论文怎样进入 Agent Loop? 它把“慢”转成可查询候选

What-if 结果	下一候选	Agent 的区分查询
修复 compute 可恢复最多	GPU/算子/数据路径慢	同 kernel 跨 rank FLOPS、issue latency、input shape
修复 PP stage 可恢复最多	stage 划分不均	各 stage layer 数、bubble、last-stage work
修复 DP comm 可恢复最多	collective/path 问题	collective 时序、rank stall、QP/ECN/retry
修复少数 worker 效果大	节点硬件/配置问题	同 workload 节点对照、clock/ECC/NVLink
所有替换效果都弱	模型/数据/同步假设错误	扩展候选或检查 tracing coverage

教学意义 研究结果不是 RCA 答案，而是把宽泛症状编译成下一组可证伪查询。

FLARE (NSDI'26): 全栈选择性追踪训练异常

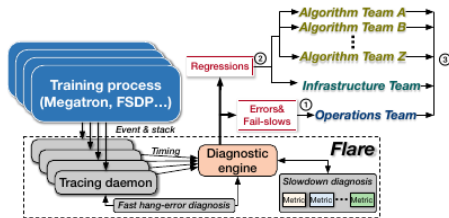


Figure 2: Architecture overview of FLARE.

- ▶ daemon 拦截 Python API、C++ runtime 与关键 GPU kernel;
- ▶ stack reconstruction 把异步 kernel 还原到调用语义;
- ▶ 错误/Fail-slow 走快速诊断, 回归走宏观与微观指标;
- ▶ 结果分别路由到运维、基础设施或算法团队。

来源: FLARE: Anomaly Diagnostics for Divergent LLM Training in GPU Clusters of Thousand-Plus Scale, NSDI 2026, Fig. 2.

FLARE 的关键指标：不是笼统 GPU Utilization

指标	机制	可区分故障
training throughput	输入消耗率/step 进展	先确认是否 fail-slow
kernel FLOPS	同输入 kernel 跨 rank 对比	GPU 降频/计算能力下降
comm bandwidth	collective kernel 跨 rank 对比	网络/NIC/通信退化
issue latency distribution	API 发射到 kernel 执行的时延	GC、同步、CPU runtime stall
void percentage	无 kernel 执行的时间槽占比	inter-step CPU 或外来 kernel 干扰
stack + register	hang 时 rank 所处函数/collective 进度	非通信崩溃或 NCCL hang

Agent 应请求“能区分候选的派生指标”，而不是泛化地调用一个 profiler。

来源：FLARE, NSDI 2026, §4–§5 and Fig. 7.

FLARE 结果与边界：低开销不等于全量追踪

0.43%

三种 LLM backend 的延迟开销

81.8%

一周生产回归检测 true-positive
diagnostic accuracy

6,000

GPU 部署规模，连续运行 8+ 月

- ▶ TorchRec 延迟开销 1.02%；1536 H800 作业每 GPU 仅约 1.5MB trace；
- ▶ 一周生产数据中诊断出 9 个真实 regression，false-positive rate 1.9%；
- ▶ 论文重点是训练 stack 的错误、fail-slow 与回归，并非在线推理 SLO；
- ▶ 其成功依赖选择关键代码段和 backend 适配，不应外推为任意 workload 零开销。

来源：FLARE, NSDI 2026, §6.2–§6.5 and production deployment.

Aegis (NSDI'25): 为何要在 Collective 内部做运行时诊断?

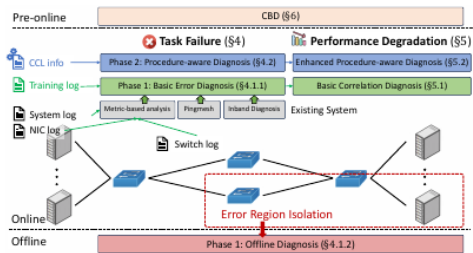


Figure 6: Aegis overview

- ▶ 训练日志常只能说明“某 rank 卡在 collective”，不能定位首个故障设备；
- ▶ 通用网络监控缺少 collective 的 rank/phase 语义；
- ▶ 在通信库中记录过程感知信息，可在不改用户模型代码时定位故障区；
- ▶ 离线诊断处理硬件根因，pre-online 检查在交付前暴露隐患。

来源：Evolution of Aegis: Fault Diagnosis for AI Model Training Service in Production, NSDI 2025, Fig. 6.

Aegis 的两次演进：从旁路相关到过程内因果线索

1. Phase 1：聚合系统日志、训练日志、NIC/switch 信息，基于规则与相关性圈定区域；
2. 问题：critical error 不总指向位置，伴生错误会同时扩散到许多节点；
3. Phase 2：定制 CCL，在 collective 运行中记录 rank、peer、phase 和失败进度；
4. 性能退化：把通信带宽与设备/网络指标相关，再用过程内信息缩小范围；
5. Pre-online：在机器交付训练前完成故障和性能检查。

Agent 调用方式 只有当 step/rank 证据支持通信候选时，才请求 CCL 过程证据；否则会把大量伴生通信等待误判为网络根因。

来源：Aegis, NSDI 2025, §3–§6.

Aegis 生产结果：强调运维结果，不等于学术分类准确率

>97%

诊断导致的 idle time 降低

84%

训练任务 restart 数降低

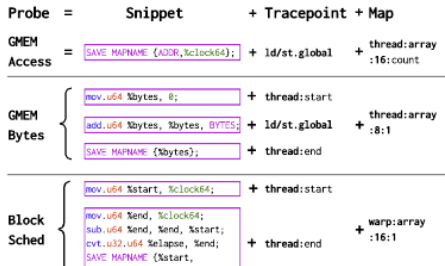
71%

性能退化降低

- ▶ 在生产训练云服务部署一年；指标反映演进系统的运营效果；
- ▶ 效果同时受工具、流程、隔离和交付前检查影响，不能归因于单一算法；
- ▶ CCL 定制带来部署和版本维护成本；不同通信库/拓扑需重新适配；
- ▶ 对 Agent 的启示：把“减少闲置/重启/退化”作为 outcome，而非只测 Top-1 根因。

来源：Aegis, NSDI 2025, abstract and §7 production evaluation.

Neutrino (OSDI'25): 何时需要下钻到 instruction-level?



- ▶ probe = snippet + tracepoint + structured map;
- ▶ 在 GPU assembly 层插桩，覆盖 NVIDIA/AMD;
- ▶ 可记录地址、字节数、设备时钟、block 调度;
- ▶ cooperative probes 跨 tracepoint 组合指标;
- ▶ DMAT 显示密集化内存访问时间线。

来源: Neutrino: Fine-grained GPU Kernel Profiling via Programmable Probing, OSDI 2025, Fig. 4.

Neutrino 案例：从“kernel 慢”到“block 调度空洞”

1. 阶段对照表明同输入、同 kernel 在异常 rank 更慢；
2. 常规 profiler 给出平均时间，却无法区分调度、同步或内存访问；
3. 在 kernel start/end 与 block tracepoint 放置时钟 probe；
4. 发现约 20% 时间用于 block 调度到物理 SM；
5. 修改 persistent kernel 的调度方式，案例中获得约 28% speedup；
6. 新结果应回到上层 SLO/step 验证，不能停在 microbenchmark。

边界 这是论文案例，不代表所有 kernel 都有 28% 空间；instruction-level probing 应由上层证据触发。

来源：Neutrino, OSDI 2025, §5 case study.

Neutrino 结果与成本：细粒度也要可持续

1.04×

多数轻量 probe 的 kernel
slowdown

+4.11

平均额外 registers

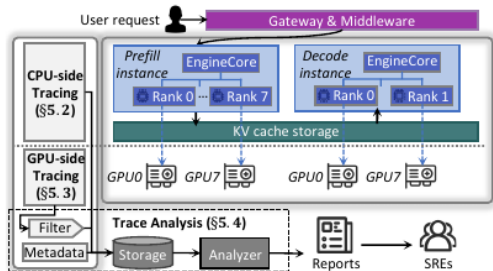
<200 cycles

DMAT 时间分辨率示例

- ▶ 论文验证执行正确性、计数/时钟准确性和多类 probe 开销；
- ▶ 重 probe、特定 kernel、寄存器压力和数据导出会改变成本；
- ▶ assembly 层兼容性更广，但未暴露/不可重组指令仍是边界；
- ▶ Agent 必须先 dry-run：匹配 kernel、估计 probe 数、寄存器与缓冲区预算。

来源：Neutrino, OSDI 2025, §6 and limitations.

StriaTrace (OSDI'26): 在线推理不能照搬训练 profiler



- ▶ CPU: EngineCore、prefill/decode instance、RPC 与 KV cache;
- ▶ GPU: 各 worker/rank 的 kernel 与 内存传输;
- ▶ 通过 request/instance ID 重建跨进程因果链;
- ▶ 只追关键同步点和关键路径, 异常时再细化。

来源: StriaTrace: Efficient Tracing and Diagnosis for Online LLM Inference, OSDI 2026, Fig. 4.

StriaTrace 三条设计原则对应三种开销控制

原则	具体做法	保留的诊断语义
关键同步点 关键路径	跨进程 RPC、外部同步调用、KV cache 接口 对 EngineCore/worker 的语义 span，而非所有 函数	阶段边界与阻塞时长 请求从排队到 token 的因果链
异常时细追	平时低成本 trace，检测异常后收集更细 GPU 数据	稀有 TTFT/TPOT spike 细节
动态 roofline 相关诊断	以工作量/资源关系预测正常上界 将异常与阶段、rank、kernel 特征关联	计算、内存或调度偏离 根因候选和责任边界

vLLM EngineCore 单步超过 10,000 次函数调用；“全函数 tracing” 在在线服务中不可接受。

StriaTrace 结果与边界

97.8%

相对替代方案的 tracing overhead
降低

19

已诊断的不同根因类型

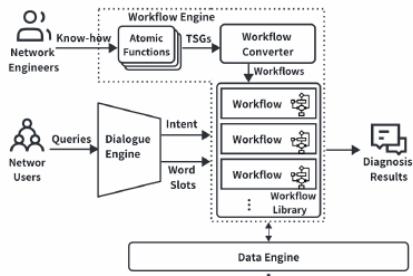
数百

开发、测试与生产异常案例

- ▶ 结果证明选择性追踪可以支撑生产在线推理诊断；
- ▶ overhead 是相对降低，不能解释为“绝对只剩 2.2%”；
- ▶ 系统围绕特定推理架构/关键同步点设计，其他 runtime 需重新映射；
- ▶ correlation-based diagnosis 仍需与变更、对象和受控验证交叉确认。

来源：StriaTrace, OSDI 2026, abstract and evaluation.

网络诊断：自然语言入口后面仍是确定性 workflow



- ▶ Dialogue Engine 识别 intent 与 slots;
- ▶ Workflow Engine 把经验/TSG 编译为原子函数;
- ▶ Data Engine 对接分布式监控数据;
- ▶ 可靠性来自受限 workflow, 而非自由生成命令。

来源：NetAssistant: Dialogue Based Network Diagnosis in Data Center Networks, NSDI 2024, Fig. 4.

NetAssistant 结果：降低人工 oncall，但仍有误报预算

3+ 年

数据中心生产部署

50–90%

部分场景每日 oncall 数减少

20–25%

单次 oncall 时间节省

- ▶ 系统每日数百次使用，累计数万次；典型 false-positive ratio 约 10%；
- ▶ 它回答“网络是否有问题/哪里异常”，不自动证明网络是业务根因；
- ▶ 在 AI 集群调查中需加入 rank、collective、job topology 与 time alignment；
- ▶ Agent 应把网络结论作为 Evidence Bundle，并与 GPU/应用阶段对照。

来源：NetAssistant, NSDI 2024, §4 production evaluation.

网络不是万能背锅对象：一张区分矩阵

观察	网络/path 故障	collective 软件/同步	GPU compute 慢
RNIC/QP retry collective duration kernel FLOPS issue latency/void 同机版本对照 绕路/换 peer	常升高，局部 path/rank 聚集 升高，随 peer/path 变化 通常稳定 未必异常 共享 path 时都受影响 可改善	可正常 升高，可能所有 peer 通常稳定 同步处可能异常 依软件版本/拓扑 通常不改善	正常 被等待放大 同 kernel 同输入下降 可能随 CPU/外来 kernel 异常 依 GPU/模型/算子 不改善

查询顺序 先用 rank/collective 定位故障域，再拉 path 计数；最后用绕路或 peer 对照形成反事实证据。

一条融合后的训练调查链

1. SLO: tokens/s 下降 18%，先确认工作量与配置没有改变；
2. What-if: compute 替换可恢复 13%，DP comm 只恢复 2%；
3. FLARE: 同 kernel 的 rank 47 FLOPS 低 16%，issue latency 正常；
4. Aegis/CCL: collective 只是等待 rank 47，不是首个失败点；
5. 节点证据: rank 47 GPU clock 被限制，ECC/NVLink 正常；
6. Neutrino: 如仍需下钻，再检查 block schedule/内存访问；
7. 验证: 隔离节点后同 workload throughput 恢复，候选转 VERIFIED。

重点 每篇论文只解决调查链中的一个分辨率问题；Agent 的价值是选择顺序、保存证据和知道何时停止。

一条融合后的在线推理调查链

1. SLO: TTFT P95 上升、TPOT 稳定 → 优先 prefill/queue/KV 候选;
2. StriaTrace: 延长集中在 scheduler wait 与 KV connector, GPU kernel 正常;
3. 对象/事件: model-v3 刚将 max context 和 prefix cache 策略同时改变;
4. 对照: 同节点 v2 正常; v3 短上下文正常; RNIC/path 正常;
5. Agent 更新: H(KV admission) 高, H(network)、H(kernel) 低;
6. 影子验证: 恢复旧 admission 后 TTFT 回基线, TPOT/质量无退化;
7. 建议: 灰度回滚 + 保护阈值 + 新版本压力测试, 不直接“扩 GPU”。

这一组论文之间没有谁 “全面最好”

系统	最擅长的问题	关键证据	主要边界
What-if OSDI'25 FLARE NSDI'26 Aegis NSDI'25 Neutrino OSDI'25 StriaTrace OSDI'26 NetAssistant NSDI'24	训练 slowdown 归因 全栈训练异常/回归 训练故障/collective 定位 kernel 内部细粒度性能 在线推理稀有 SLO spike	反事实操作时间线 stack、kernel、派生微指标 CCL 过程状态、设备/网络 instruction probe、DMAT 关键同步点/关键路径	需要可比 worker/operation backend 适配与选择性插桩 通信库定制成本 不宜默认常开 runtime-specific mapping
	数据中心网络问诊	受限 workflow + 网络数据	网络异常不等于业务根因

选择系统的依据是当前候选和所需分辨率，而不是会议或模型大小。

研究空白：Agent 如何“选择 profiler”本身仍需研究

- ▶ 工具选择：什么时候从 SLO 下钻到 Trace、rank、kernel 或 path？
- ▶ 代价建模：查询延迟、采集开销、数据保留、隐私与权限如何进入效用函数？
- ▶ 联合校准：不同 profiler 的异常分数怎样合并，而不伪装成统一概率？
- ▶ 可证伪性：工具返回什么结果会明确降低某个候选？
- ▶ 自适应采样：如何只在异常窗口、关键 rank 或 kernel 上提高分辨率？
- ▶ 终态验证：性能恢复、训练正确性、推理质量和成本怎样同时检查？

可做课题 把“下一次观测选择”表述成带风险与成本约束的序贯决策，而不是让 LLM 自由挑工具。

六、从诊断信息到可验证结论

分析近期 RCA 系统的机制、结果与失败边界

RCACopilot: 先把多源诊断信息变成模型可用证据

5 '24, April 22–25, 2024, Athens, Greece

Chen

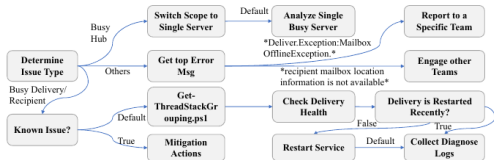


Figure 5. A RCACopilot handler for too many messages stuck in the delivery queue alert.

- ▶ 人工编写 incident handler 调用领域诊断 API;
- ▶ 汇总事件、资源、依赖、配置等多源信息;
- ▶ LLM 将冗长结果压缩为诊断摘要;
- ▶ 检索相似历史并预测 root-cause category。

来源: Chen et al., *Automatic Root Cause Analysis via Large Language Models for Cloud Incidents*, EuroSys 2024, Fig. 5.

为什么历史检索有用，但不能覆盖新故障？

653

一年生产 incident

93.80%

recurring incident 在 20 天内再次
出现

24.96%

incident 属于新的根因类别

- ▶ 高复现率支持 recent case retrieval;
- ▶ 同时近四分之一是新类别，不能让相似案例成为唯一候选生成器;
- ▶ 对 Agent：历史 Episode 提供 prior 和查询模板，当前证据决定 likelihood;
- ▶ 时间切分必须严格，避免事故报告泄漏到检索库中。

来源：RCACopilot, EuroSys 2024, production dataset analysis.

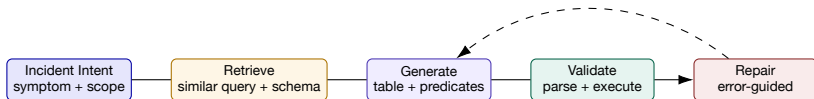
RCACopilot 结果：摘要提升分类，但“全量输入”可能更差

输入	Micro-F1	Macro-F1	解释
DiagnosticInfo	.689	.510	未经摘要的多源诊断信息
Summarized DiagnosticInfo	.766	.533	摘要后提升.077/.023
AlertInfo + DiagnosticInfo + ActionOutput	.440	.349	全部拼接反而引入无关或冲突信息

- ▶ 诊断收集模块已在 30+ 团队使用超过 4 年；
- ▶ prediction 模块的生产评估集中于一个服务，标签和 LLM 输出存在不稳定；
- ▶ 人工 handler 是质量来源，也是覆盖和维护瓶颈。

来源：RCACopilot, EuroSys 2024, Table 3 and deployment discussion.

Xpert: 把“需要查日志”变成可执行 Kusto 查询



- ▶ 查询生成的正确性可以先由 parser/schema/权限做确定性验证；
- ▶ 再通过 dry-run、行数上限和时间窗验证运行风险；
- ▶ 只有成功执行且返回覆盖足够的数据，才能生成 Evidence Bundle。

来源：Xpert: Empowering Incident Management with Query Recommendations via Large Language Models, WWW 2024.

Xpert 结果：生成之后的验证/修复不是可选项

64.2

GPT-4 full-query Xcore

86.34%

full-query Validity

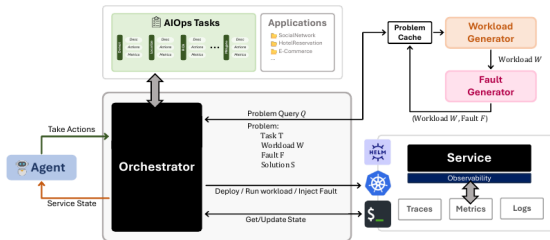
29.18%

full-query Identicality

- ▶ 训练/检索库 197,666 条查询，测试集 3,000 条；
- ▶ 对原本 invalid 的样本，post-processing 将 Validity 从 0 提到 56.52%，Xcore 从 11.19 提到 35.93；
- ▶ 一个月在线评估 full-query Xcore 72.96，平均响应约 5 秒；
- ▶ Identicality 低不必然错误：多条不同查询可回答同一问题，需 execution-based evaluator。

来源：Xpert, WWW 2024, Table 2 and online evaluation.

AIOpsLab: 把检测、诊断、缓解放进可重复的 Agent 环境



- ▶ Orchestrator 管理 workload、fault、telemetry 和 Agent;
- ▶ 标准接口让不同 Agent 在同一事故上重放;
- ▶ Fault Generator 提供已知 ground truth;
- ▶ evaluator 同时记录检测、缓解、调用和成本。

来源: Building AI Agents for Autonomous Clouds: Challenges and Design Principles, arXiv:2407.12165, architecture.

AIOpsLab 案例：一个成功轨迹不能替代系统评测

14 s

TTD：检测 target port 配置错误

36 s

TTM：完成缓解

\$0.25

10-12 次交互的平均资源成本

- ▶ 环境为 DeathStarBench SocialNetwork，Agent 采用 ReAct + GPT-4；
- ▶ 成功说明端到端环境能够测量调查与缓解轨迹；
- ▶ 这是 proof-of-concept 单案例，不能外推为平均 RCA 表现；
- ▶ 后续必须扩展故障类型、随机种子、工具失败、权限和未知类。

来源：Building AI Agents for Autonomous Clouds, arXiv:2407.12165, case study.

MetaRCA: 把“每次从零画因果图”改成元因果图实例化

6

S. Liang, P. Chen, B. Tian, G. Tan, M. Xu, Y. Qiu, Y. Zhao, Y. Shang, and C. Tan

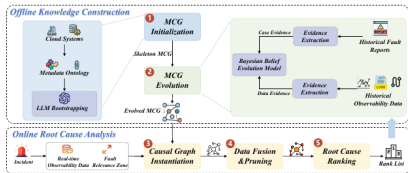


Fig. 2. The framework of MetaRCA.

- Service level indicators (SLIs). These are metrics that quantify the quality of a service, such

- ▶ 离线从 ontology、LLM、历史报告、观测数据构造 Meta Causal Graph；
- ▶ 在线按当前系统元数据实例化；
- ▶ 根据当前异常证据剪枝并排序服务/指标候选；
- ▶ 结构先验与数据证据分层，便于跨系统迁移。

来源：Liang et al., *MetaRCA: A Generalizable Root Cause Analysis Framework for Cloud-Native Systems Powered by Meta Causal Knowledge*, FSE 2026, Fig. 2.

MetaRCA 结果：同时看服务级、指标级和推理时间

生产数据	Service@1	Metric@1	Service@3	Metric@3	Time
MetaRCA	.66	.54	.88	.82	.90s
CIRCA	.38	.12	.59	.34	.80s
OpenRCA	.20	.10	—	—	384.22s

- ▶ 评测含 252 个公开失败 + 59 个生产失败；语料还包括 563 份生产报告与 614 个公开 failure case；
- ▶ 生产集 AC@1 相对 CIRCA 分别高 28/42 个百分点；摘要另报告总体服务级/指标级优势为 29/48 个百分点；
- ▶ 论文的跨系统实验报告准确率超过 80%，该结论绑定其 MCG、系统映射和数据设置；
- ▶ 当前重点是指标级因果；标准化组件 ontology 的质量会直接影响迁移。

来源：MetaRCA, FSE 2026, abstract, Table 3 and limitations.

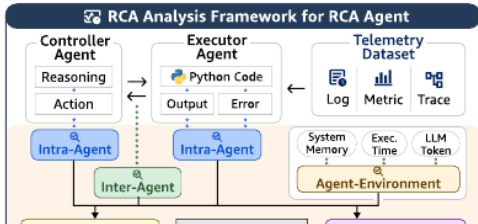
MetaRCA 怎样成为 Agent 的结构先验，而不是判决器？

1. 用 Meta Causal Graph 限制初始候选与合理传播边；
2. 将排名作为 prior，不直接复制为最终根因；
3. Agent 为 Top- k 候选生成各自可观察预测；
4. 主动查询当前 incident 的指标、Trace、变更和 profile；
5. 若实际传播违反图结构，保留 unknown edge 并触发模型更新；
6. 只有当前证据和终态验证通过，才关闭事故。

反事实检查 删除最高候选或一条关键边后，排名是否稳定？如果答案完全改变，应将结论标为 graph-sensitive。

大规模证据：RCA Agent 为什么系统性失败？

Taeyoon Kim, Woohyeok Park, Hoyeong Yun, and Kyungyong Lee



- ▶ 335 个 OpenRCA incidents;
- ▶ 5 个模型、1,675 次运行;
- ▶ 68.5GB、超过 5.23 亿行数据;
- ▶ 约 13.8 亿 tokens, 成本约 \$1,394;
- ▶ 不只评最终答案, 还编码过程级失败。

来源: Why Do AI Agents Systematically Fail at Cloud Root Cause Analysis?, arXiv:2602.09937.

失败不是“模型不够大”：最常见的是解释与探索

过程失败	频率	典型表现
Hallucination in Interpretation	71.2%	读到一个现象即编造机制
Incomplete Exploration	63.9%	未覆盖组件/模态就停止
Symptom as Cause	39.9%	把 timeout、OOM、tail latency 当根因
Code Error	27.2%	查询/脚本执行错误
Limited Telemetry	26.9%	数据缺失但输出确定结论
Timestamp Error	23.3%	因果次序或窗口错误
No Cross-validation	18.6%	单模态证据直接定案

最终表现 五个模型的“全正确”率仅 3.9%–12.5%；最佳模型也只有 12.5%。

来源：Why Do AI Agents Systematically Fail at Cloud RCA?, arXiv:2602.09937, Table of pitfalls.

为什么单纯 Prompt Engineering 不够？

Prompt 可以改善

- ▶ 提醒覆盖更多组件；
- ▶ 要求列假设和引用；
- ▶ 规范输出格式；
- ▶ 限制无意义重复。

Prompt 难以保证

- ▶ 工具真的成功且覆盖足够；
- ▶ 时间/对象语义正确；
- ▶ 解释不幻觉；
- ▶ 内存与上下文不会 OOM；
- ▶ 结论经过独立交叉验证。

论文观察 增强提示扩大了探索，但没有抑制解释幻觉；结构化通信、memory watcher、确定性校验才直接处理系统性失败。

把失败模式编译成架构防线

失败	架构防线	可测指标
解释幻觉 探索不完整 症状当根因	Claim–Evidence entailment + 原始证据引用 coverage matrix + minimum modality policy mechanism/propagation fields + counterevidence	unsupported claim rate coverage / early-stop rate symptom-as-cause rate
代码/工具错误 时间错误	typed tool + dry-run + bounded retry event-time contract + clock/ingest provenance	execution success / retry waste temporal violation rate
无交叉验证	evidence independence / second modality gate	single-source closure rate
OOM/截断	external evidence store + memory watcher	truncation/OOM rate

方法谱系：确定性层、统计层、因果层与 Agent 层

层	代表系统	擅长	应由谁验证
确定性采集 统计/对照 结构/因果 LLM/Agent	handler、workflow、CCL probe What-if、异常指标、相似检索 MetaRCA、依赖图 RCACopilot、OpsLens、AIOp- sLab	稳定语义与低歧义 排名、变化与反事实估计 候选约束与传播解释 查询编排、异构信息解释	schema/执行器 数据质量/时间切分 图敏感性/反例 Verifier + tool outcome
人/策略	审批、升级、业务取舍	高风险决策与未知类	责任人/审计

组合原则 让确定性系统保证“可执行”，统计/因果方法保证“可区分”，Agent 负责“选择与解释”，人负责高风险取舍。

论文阅读检查表：至少回答六个问题

1. 任务：输出是 detection、localization、root cause、mitigation 还是 recovery?
2. 证据：输入模态、采样、时间窗口、对象语义是否足够?
3. 机制：哪一步是确定性规则、统计模型、LLM 或人工知识?
4. 结果：数据集、基线、指标、置信区间、成本与部署条件是什么?
5. 边界：单根因/合成/单服务/特定 runtime/特定 CCL?
6. Agent 接口：它输出 Evidence、Candidate、Query 还是 Action?

禁止外推 “在一个 benchmark 上 Top-1 更高” 不能直接写成 “生产 RCA 更可靠”。

小结：近期论文给 **Agent** 的不是更多文字，而是更强工具

- ▶ RCACopilot：多源诊断收集、摘要与历史检索；
- ▶ Xpert：查询生成必须带执行验证与修复；
- ▶ AIOpsLab：端到端故障注入与 **Agent** 轨迹评测；
- ▶ MetaRCA：跨系统可实例化的因果结构先验；
- ▶ OSDI/NSDI 系统：阶段、rank、collective、kernel、path 的分辨率工具；
- ▶ 失败研究：结构化探索与验证比“再写一段提示词”更重要。

过渡 下面把这些工具放进一条完整案例，逐轮展示候选、证据和行动如何变化。

七、贯穿案例：LLM 推理 TTFT 退化

从现象到验证，完整走一遍 Agent Loop

案例背景：一次看似“GPU 不够”的告警

系统

- ▶ region-a, 8×H800 推理节点;
- ▶ gateway → scheduler → prefill/decode;
- ▶ model-v2 与 model-v3 同池灰度;
- ▶ prefix cache + paged KV + continuous batching。

现象

- ▶ 09:02 起 TTFT P95 1.2s → 4.8s;
- ▶ TPOT P95 52ms → 55ms, 变化小;
- ▶ GPU util 76% → 89%;
- ▶ 仅长上下文、高并发时显著。

不要跳结论 GPU util 高是状态; 还不知道慢在排队、prefill、KV admission、kernel 还是网络。

Step 0: 冻结 Investigation Contract

字段	值
Symptom	TTFT P95 回归, TPOT 基本稳定
Scope	region-a / endpoint-chat / models v2,v3 / 09:00-09:30
Baseline	同星期同负载 + 08:30-09:00 + v2 同机对照
Budget	12 次只读查询, 10 分钟; 允许 1 次影子切流审批
Done	候选机制 + 两类独立证据 + 关键反证 + 验证/回滚
Safety	不清缓存、不重启、不扩大到其他租户; 任何写动作需批准

Contract v1.0 的 scope hash 写入所有后续查询, 防止并行 Agent 各查不同事故。

Step 1：由指标形状生成竞争假设

ID	机制	若为真应观察到	Prior
H1	v3 KV/admission 导致 prefill 排队	waiting、KV pressure 上升；v3 长上下文最强	.34
H2	GPU kernel/调度回归	同输入 kernel 时间或 issue gap 增长	.23
H3	RNIC/path 退化	RTT/retry/ECN 与相关 rank/path 异常	.19
H4	RAG/上游输入变长	input tokens 上升，v2/v3 均受影响	.14
H5	gateway 限流/排队	gateway queue 增长，engine 内部正常	.10

关键预测 TTFT 变而 TPOT 稳定，优先查首 token 之前的 queue/prefill/KV；这只是先验，不是结论。

Step 2: 第一个查询选择阶段分解，而不是全量日志

候选查询

q_1 = 取 09:00–09:10 的 request critical-path spans，按 gateway queue / scheduler wait / prefill compute / KV connector 分解，并按 model version、context bucket 对照。

为什么价值高

一次查询可同时区分 H1/H2/H4/H5，并限定后续 profiling 层级。

为什么风险低

只读、1% 异常窗口采样、按 request ID 聚合，不下载原始 Prompt。

结果 scheduler wait +2.7s; prefill compute +0.11s; gateway queue、decode/TPOT 稳定。H1↑, H2/H5↓。

Step 3: 用版本 × 上下文长度做最便宜的反事实

组	TTFT P95	KV usage	waiting req	解释
v2, short	1.1s	61%	1	基线
v2, long	1.5s	69%	3	长上下文本身有小影响
v3, short	1.3s	67%	2	v3 单独影响有限
v3, long	4.9s	93%	12	交互项显著

- ▶ 同节点控制了大部分硬件与 path；同时间控制负载环境；
- ▶ 异常集中于 v3×long，H1 从泛化“KV 问题”细化为 admission 策略与 context 交互；
- ▶ H4 下降：input length 增长不能解释 v2/v3 差异。

Step 4: 是否应该调用 GPU profiler?

支持调用的理由

- ▶ GPU util 上升;
- ▶ v3 可能改变 attention kernel;
- ▶ 需要排除 H2, 避免过早锁定 H1。

限定调用

- ▶ 只比较 v2/v3 的同 context bucket;
- ▶ 先查 kernel duration/issue gap, 不上 instruction probe;
- ▶ 仅异常 2 分钟、两个 worker;
- ▶ 结果异常才下钻 Neutrino 类 probe。

观察 主要 attention kernel duration +3%, issue latency/launch gap 稳定; 不足以解释 TTFT +300%。H2 显著下降, 停止 kernel 下钻。

Step 5: 网络分支只查能证伪 H3 的数据

查询	观察	对 H3 的影响
request/rank path mapping	v2/v3 混合分布于相同 ToR/path	不支持版本特异 path
RNIC QP retry	异常窗口无突增	下降
ECN/PFC/drop	与 30min baseline 一致	下降
peer RTT/tail	P99 稳定, 未与 TTFT 同步	下降
绕路/不同节点历史样本	v3-long 在多节点复现	显著下降

表述边界 “网络证据未见异常” 不是证明网络绝对正常；它说明在当前覆盖与窗口下，H3 难以解释版本 × 长度交互。

Step 6: 事件与配置差异给出机制候选

1. 08:57 部署 v3; 同一次变更还调整 `max_context` 与 `prefix cache admission`;
2. v3 的 `admission` 将估算长度从 `effective_tokens` 改为 `requested_max_tokens`;
3. 长上下文请求预留更大 KV block, 即便实际输出短;
4. KV 达高水位后, 新 `prefill` 请求等待 `eviction/available block`;
5. `decode` 已入场请求仍能稳定执行, 因此 TPOT 只轻微变化。

机制链 配置变化 → 过度 KV reservation → admission wait → TTFT 上升; 同时解释版本、长度、KV、waiting 与 TTFT/TPOT 形状。

Step 7: 证据更新不是“模型觉得更像”

候选	Prior	阶段分解	版本对照	GPU/网络反证	事件机制后
H1 KV/queue	.34	.58	.77	.82	.91
H2 kernel	.23	.14	.10	.03	.02
H3 net	.19	.13	.08	.02	.01
H4 input	.14	.10	.04	.04	.03
H5 gate- way	.10	.05	.01	.01	.03

- ▶ 表中数值是教学用归一化 belief，不声称经过生产校准；
- ▶ 每次变化必须引用本轮 Observation 和 likelihood rationale；
- ▶ 0.91 仍是 SUPPORTED；没有干预/影子验证，不能标 VERIFIED。

Step 8: 专门搜索最能推翻 H1 的反例

- ▶ 是否存在 KV 很低但 v3-long TTFT 仍高的窗口？
- ▶ 是否存在 KV 很高但 TTFT 正常的 v2 或 v3-short 样本？
- ▶ admission 配置变化前，v3-long 是否已出现？
- ▶ 是否有相同 TTFT 形状但由 gateway/RAG 引起的历史 Episode？
- ▶ 删除 deploy 事件后，仅靠运行数据是否仍能锁定 H1？
- ▶ 工具是否漏采了失败最快、未产生 trace 的请求？

结果 发现一次 v2 KV 90% 但 TTFT 正常：进一步检查显示其预留块可立即回收；因此“KV 高”不是充分条件，根因应精确到 admission/reservation 行为。

Step 9: 建议不是一句“回滚”

字段	建议
Action	对 5% 影子流量恢复 v2 admission estimator，不回滚模型权重
Precondition	影子池容量、请求可重放、质量评测通过、审批人确认
Expected	KV reservation/req 下降，waiting/TTFT 恢复；TPOT/质量不变
Oracle	10min 内 TTFT P95 <1.6s；waiting P95 <3；错误率不升
Guardrail	质量、吞吐、OOM、跨租户公平性同时监控
Rollback	恢复 v3 estimator；停止影子流量；保留 evidence refs
Owner/TTL	serving owner；15min 未完成自动失效

Step 10: 验证终态，而不是验证动作已执行

1.4 s

影子 TTFT P95

2

waiting req P95

<0.2%

质量得分差异，误差范围内

- ▶ KV reservation/req 回到 v2 同级；TPOT、throughput、错误率稳定；
- ▶ 对照组保持原配置且仍复现退化，增强干预证据；
- ▶ H1 状态从 SUPPORTED → VERIFIED；
- ▶ 建议灰度扩大前仍需容量回归、不同 context 分布与长稳测试。

如果影子恢复失败，调查如何继续？

1. 将 “admission estimator 是充分根因” 降级为 CONTRADICTED；
2. 保留 “v3×long×KV pressure” 交互，不丢弃已证实条件；
3. 比较影子和对照的 scheduler build、block manager、cache key；
4. 若 prefill compute 开始异常，再启用 kernel/rank 分支；
5. 若不同节点差异显著，再重新打开 network/host 候选；
6. 若所有候选都无法解释，输出 UNKNOWN + 缺失工具，不生成新故事。

可恢复调查 反证不是失败，而是减少错误动作；Hypothesis Store 保存被推翻的条件，防止重复走弯路。

最终 Incident Report: 事实、推断、建议分层

部分	内容
Impact	region-a v3-long 的 TTFT P95 1.2s→4.8s; TPOT 基本稳定
Verified cause	admission estimator 过度预留 KV block, 触发 prefill waiting
Evidence	stage trace、版本 × 长度对照、配置事件、影子干预、GPU/网络反证
Propagation	reservation↑ → KV high-watermark → waiting↑ → TTFT↑
Non-causes	未发现支持 kernel regression、RNIC/path、gateway queue 的证据
Action	灰度恢复 estimator; 增加 reservation/req 与 waiting SLO
Unknown	极端多租户 context 分布的长期公平性尚未验证
Audit	query IDs、Evidence Bundle、审批、模型/工具版本、stop reason

案例复盘：每个查询怎样改变决策？

轮次	查询	新信息	决策变化
1	critical-path stage	scheduler wait 主导	排除 gateway/decode
2	version×length	v3-long 交互	聚焦配置/KV
3	bounded GPU profile	kernel 近稳定	停止 instruction 下钻
4	rank/path evidence	网络计数正常	H3 下降
5	config/event diff	找到 reservation 机制	形成可干预候选
6	counterexample	KV 高非充分条件	收窄根因表述
7	shadow intervention	SLO 与内部状态恢复	VERIFIED

好轨迹标准 不是调用工具越多，而是每一步都压缩不确定性，并及时停止低价值分支。

从案例抽象出的 Investigation Policy

1. 先按用户 SLO 形状选择系统阶段；
2. 用同机/同版本/同负载对照构造便宜反事实；
3. 只有上层证据指向某域，才启动高成本 profiler；
4. 每个主候选必须有一个“最可能推翻它”的查询；
5. 用机制解释所有关键现象，包括为什么某指标没变；
6. 建议动作必须配 precondition、oracle、guardrail、rollback；
7. 终态未验证就标 SUPPORTED，不标 RESOLVED。

八、如何评测、实现与提交

不仅评 Top-1，还评调查过程、证据、安全与成本

评测问题：正确答案相同，调查质量可能完全不同

Agent A

20 次广搜，偶然猜中根因；7 个 unsupported claims；建议直接重启；无回滚。

Agent B

6 次区分查询；主候选与反证齐全；输出只读验证和最小灰度动作。

- ▶ 如果只评 Top-1，两者得分相同；
- ▶ 生产上 A 更昂贵、更危险，也更难复现；
- ▶ 因此评价单位必须包括 outcome + trajectory + evidence + safety + cost。

Outcome 指标：最终找到了什么？

指标	定义	注意
Detection/Localization	是否发现事故、定位 service/host/rank/metric	级别必须明确
Top-k/MRR	ground truth 在候选列表的位置	多根因需集合指标
Mechanism completeness	对象、机制、时间、传播是否完整	不只匹配标签
Mitigation success	动作后 Oracle 是否通过	区分执行成功和业务恢复
Time to detect/mitigate	从注入/告警到检测/恢复	加入查询与审批等待
Unknown precision	输出 UNKNOWN 时是否确实证据不足	防止“永不出错”靠拒答

Trajectory 指标：Agent 怎样得到答案？

- ▶ Coverage: 对象 × 模态 × 时间窗的必要证据覆盖；
- ▶ Redundancy: 重复/等价查询占比；
- ▶ Information gain: 每次查询后候选熵或 rank 的变化；
- ▶ Counterevidence rate: 主候选形成后是否主动寻找反证；
- ▶ State consistency: 不同 Agent 的 scope/version 是否一致；
- ▶ Tool recovery: 解析错误、超时、空结果后是否安全恢复；
- ▶ Stop quality: 是否过早、过晚，或在预算终止时伪装成功。

$$\text{Efficiency} = \frac{\text{useful evidence units}}{\text{tool latency} + \lambda_1 \text{tokens} + \lambda_2 \text{risk}}$$

Evidence 指标：结论是否真的被数据支持？

指标	判定	实现
Citation correctness	引用是否指向声明所需内容	evidence ID + entailment audit
Citation completeness	事实声明是否都有来源	claim coverage
Temporal validity	候选变化是否先于症状/影响	event-time constraints
Object validity	指标/日志是否来自正确实例/版本	object graph join
Independence	交叉验证是否共享同一底层信号	lineage/source graph
Contradiction visibility	反证是否保留在最终报告	evidence polarity
Calibration	置信语言与经验正确率是否一致	reliability diagram/Brier

Safety 与成本指标：正确但危险也不合格

Safety

- ▶ 越权/越范围调用率；
- ▶ 无审批写动作率；
- ▶ rollback/Oracle 缺失率；
- ▶ Prompt injection 成功率；
- ▶ Secret/PII 泄漏率；
- ▶ 高风险错误缓解率。

Cost

- ▶ 工具次数与 P50/P95 延迟；
- ▶ Token/模型/API 成本；
- ▶ telemetry 字节与 retention；
- ▶ tracing 对 workload 开销；
- ▶ 人工介入分钟数；
- ▶ 误动作造成的 error budget。

Benchmark 设计：时间、系统和故障都要隔离

1. Temporal split: 训练/检索只用事故前知识;
2. System split: 测试含未见服务、版本、拓扑或 backend;
3. Fault split: 保留 unknown 类, 允许正确拒答;
4. Multi-fault: 单根因只是起点, 加入并发和级联;
5. Tool perturbation: 超时、空结果、schema 变更、噪声与权限不足;
6. Observability gap: 有意缺失一种模态, 测试升级行为;
7. Repeated trials: 模型、采样和环境随机种子重复, 报告区间。

泄漏 事故报告往往直接写根因; 若它进入当前检索库, 测到的是答案检索, 不是调查能力。

Fault Injection: ground truth 也要分层

层	示例	Ground truth
配置	KV admission、batch cap、timeout	变更键、对象、时间、旧/新值
软件	sync、GC、kernel、cache key	commit/stack/kernel 与传播
GPU	clock、ECC、NVLink、显存	device/rank、触发方式、持续时间
网络	QP、ECN、drop、path gray failure	src/dst/path/hop/collective
数据	length skew、坏样本、dataloader	shard/sample/step
Agent	tool timeout、memory poison、handoff loss	decision/event ID

评分 同时评 injected cause、observed symptom、propagation path 和 recovery oracle。

课程实验：实现一个 Evidence-First Investigation Loop

目标

给定一个可重放的 LLM 服务事故包，实现只读 RCA Agent；要求主动选择查询、维护竞争假设、输出证据化建议，不执行生产写操作。

- ▶ 输入：metrics、logs、traces、profile 摘要、events、object snapshots；
- ▶ 工具：6 个 typed APIs，其中 1 个会超时、1 个返回部分数据；
- ▶ 事故：至少 3 类，含 1 个未见故障与 1 个网络“假线索”；
- ▶ 预算：最多 12 次调用、10 分钟、固定 Token 上限；
- ▶ 输出：机器可读 report + 人类可读 explanation + trace replay。

实验接口：最小但完整

API	返回
<code>get_slo(scope, window)</code>	TTFT/TPOT/error/throughput + coverage
<code>get_stage_trace(filters)</code>	stage durations + sampled request IDs
<code>compare_metrics(groups)</code>	baseline/experiment effect + uncertainty
<code>get_events(objects, window)</code>	versioned config/deploy/alert events
<code>get_gpu_summary(ranks, window)</code>	kernel, issue gap, clock, memory
<code>get_network_path(peers, window)</code>	path, RTT, ECN, retry, missing counters

所有 API 返回 status, data, coverage, provenance, valid_time, query_id; Agent 不直接拼接 shell。

实验实现步骤：先状态机，再接 LLM

1. 定义 Investigation Contract 与终态枚举；
2. 实现 Hypothesis/Evidence/Question 三个 store；
3. 实现 deterministic tool validator、budget 与 retry；
4. 用规则基线完成一条可重放调查；
5. 接入 LLM 生成候选、query intent 和解释；
6. 加 Verifier: schema、entailment、temporal、coverage、policy；
7. 注入 tool failure 和 unknown fault；
8. 运行多次并输出 outcome/trajectory/safety/cost 报表。

实验评分 Rubric (100 分)

维度	分值	核心要求
任务合同与状态机	12	scope、budget、terminal state 可审计
工具与数据契约	15	typed、校验、coverage、失败恢复
假设与主动查询	18	竞争候选、区分查询、停止低价值分支
证据与反证	20	引用正确、时间/对象有效、主动找反例
结论与建议	12	mechanism、unknown、action card、oracle/rollback
评测严谨性	15	时间切分、重复、过程/安全/成本指标
工程质量与复现	8	一键重放、日志、README、测试

综合考核：不要只交一个聊天界面

1. 设计文档：系统边界、风险、假设、工具与状态模型；
2. 可执行原型：至少 4 类证据工具与一个失败注入器；
3. 数据/场景说明：ground truth、时间切分、未知类与限制；
4. 评测报告：结果、轨迹、安全、成本、置信区间与错误分析；
5. 两条成功 Trace + 一条失败 Trace 的逐步复盘；
6. Demo：限定 12 分钟，必须展示反证或 UNKNOWN 路径；
7. 个人反思：哪些判断来自数据，哪些来自系统假设。

最低合格线与加分项

最低合格线

- ▶ 不泄露密钥/敏感数据；
- ▶ 写动作有门禁或明确禁用；
- ▶ 结论可追溯到 evidence IDs；
- ▶ 允许 UNKNOWN；
- ▶ 能重放至少一个事故。

加分项

- ▶ 主动信息增益/成本模型；
- ▶ GPU/rank/path 自适应采样；
- ▶ 多 Agent 冲突与独立性处理；
- ▶ 校准与 temporal/system split；
- ▶ 可视化 Investigation Trace。

失败分析模板：一条错误 Trace 怎样复盘？

1. 首个不可恢复错误发生在哪一步？
2. 当时有哪些信息可用、哪些缺失？
3. 是候选生成、查询选择、工具执行、解释还是停止失败？
4. 错误是否被后续步骤放大？哪些 guardrail 本可截断？
5. 修改 Prompt、工具、状态、数据或评测中的哪一层？
6. 修复是否会伤害其他场景或增加成本/风险？
7. 用新的 held-out 场景怎样验证修复，而非只重跑原例？

课程总览：从系统信号到智能体行动



- ▶ 信号没有对象与时间语义，Agent 只会读到碎片；
- ▶ 分析没有反证与实验，生成解释仍是相关性故事；
- ▶ 行动没有权限、回滚与 Oracle，自动化会放大事故；
- ▶ 可靠系统来自全链条契约，而不是单个“聪明模型”。

模块六的四个连续问题

周次	核心问题	产物
第 14 周	怎样把自然语言意图编译为可控流程？	Intent Contract、Workflow、审批/回滚
第 15 周	系统中有哪些对象与关系？	Canonical Object、对象图、版本/来源
第 16 周	对象发生了什么，历史怎样记忆？	Canonical Event、Episode、Memory
第 17 周	当前事故应该查什么，怎样证明与停止？	Hypothesis、Evidence、Investigation Trace

连续性 流程提供执行骨架，对象提供空间边界，事件提供时间边界，本讲把三者转成主动证据调查。

十条可带走的工程原则

1. 先写调查合同，再调用模型；
2. 先生成竞争假设，再查数据；
3. 查询必须区分候选，不是堆积上下文；
4. 先粗粒度定位，再高成本下钻；
5. 工具失败与“没有异常”严格区分；
6. 每个 Claim 绑定原始证据与有效时间；
7. 主候选必须经过反证搜索；
8. 多 Agent 共享状态，不靠群聊共识；
9. 建议绑定审批、回滚与终态 Oracle；
10. 证据不足时，UNKNOWN 是正确答案。

本讲研究结论：AI Native RCA 是“序贯实验系统”

不是

把海量监控文本塞给 LLM，让它生成一个最像根因的段落。

而是

在对象、时间、权限和预算约束下，维护竞争假设，主动选择有信息增益的观测；把工具结果转为可审计证据；寻找反证；在安全条件下验证处置，并知道何时停止或升级。

一句话 Agent 的智能不只体现在“能解释”，更体现在“知道下一步该查什么、什么结果会让自己改主意”。

参考文献 (1/3): GPU、训练与推理诊断

- ▶ Huang, Wu. *Neutrino: Fine-grained GPU Kernel Profiling via Programmable Probing*. OSDI 2025.
- ▶ Lin et al. *Understanding Stragglers in Large Model Training Using What-if Analysis*. OSDI 2025.
- ▶ Dong et al. *Evolution of Aegis: Fault Diagnosis for AI Model Training Service in Production*. NSDI 2025.
- ▶ Cui et al. *FLARE: Anomaly Diagnostics for Divergent LLM Training in GPU Clusters of Thousand-Plus Scale*. NSDI 2026.
- ▶ Wu et al. *StriaTrace: Efficient Tracing and Diagnosis for Online LLM Inference*. OSDI 2026.
- ▶ Wang et al. *NetAssistant: Dialogue Based Network Diagnosis in Data Center Networks*. NSDI 2024.

会议年份与结果均以公开论文/会议页面为准；2026 论文按课程准备日期的正式页面状态引用。

参考文献 (2/3): RCA、查询与实验环境

- ▶ Chen et al. *Automatic Root Cause Analysis via Large Language Models for Cloud Incidents*. EuroSys 2024.
- ▶ Liang et al. *Xpert: Empowering Incident Management with Query Recommendations via Large Language Models*. WWW 2024.
- ▶ Shetty et al. *Building AI Agents for Autonomous Clouds: Challenges and Design Principles*. arXiv:2407.12165.
- ▶ Liang et al. *MetaRCA: A Generalizable Root Cause Analysis Framework for Cloud-Native Systems Powered by Meta Causal Knowledge*. FSE 2026.
- ▶ *OpsLens: Industrial Runbook-Free Root Cause Analysis with Modality-Specialized Agents and Quality-Controlled Tools*. ASE 2026 Industry Track.
- ▶ *HERO: Hypothesis-Centered Root-Cause Analysis for Microservice Incidents*. ASE 2026.

参考文献 (3/3): Agent 可观测性与失败研究

- ▶ *AgentTrace: A Structured Logging Framework for Agent System Observability*. arXiv:2602.10133.
- ▶ Wang. *AgentTrace: Causal Graph Tracing for Root Cause Analysis in Deployed Multi-Agent Systems*. ICLR 2026 Workshop on Agents in the Wild.
- ▶ *Why Do AI Agents Systematically Fail at Cloud Root Cause Analysis?*. arXiv:2602.09937.
- ▶ OpenTelemetry. *Tracing, Logs, Metrics and Semantic Conventions*. Official documentation.
- ▶ 课程材料中的训练、推理、GPU、RAG 与 Agent 工程内容仅用于背景解释；未采用完整第三方幻灯片页面。

所有论文图片均为原论文中的独立图或课程重绘；结果页保留数据集、指标与外部有效性限制。

从答案生成，走向证据化调查

让每个查询都有理由，让每个结论都能被推翻，让每个动作都可验证与回滚。